



ALFIDO TECH

RIDING THE WAVES OF TECHNOLOGICAL
ADVANCEMENT

IRIS CLASSIFICATION USING MACHINE LEARNING

Alfido Tech Machine Learning

Internship

Submitted By

Name: Ankit Singh Bungla

Internship: Machine Learning Internship

Candidate ID: BS/REG/122962

Organization: Alfido Tech

Project: Iris Classification using Machine Learning

Submission Date: 20-07-2026

CERTIFICATE

I hereby declare that the project entitled “Iris Flower Classification using Machine Learning” has been completed by me during my Machine Learning Internship at Alfido Tech.

The work presented in this report is based on my own implementation and understanding of machine learning concepts, and all resources used have been properly acknowledged.

Ankit Singh Bungla, Machine Learning Intern, Alfido Tech

Acknowledgement

I sincerely thank Alfido Tech for providing me with the opportunity to complete this internship and gain practical experience in machine learning. I also acknowledge the support of open-source tools such as Scikit-learn, Pandas, NumPy, Matplotlib, and Seaborn, which were used throughout this project.

Table of Contents

1. Introduction
2. Project Objectives
3. Dataset Description
4. Technology Stack
5. Exploratory Data Analysis (EDA)
6. Data Preprocessing
7. Feature Engineering
8. Machine Learning Models
9. Model Evaluation
10. Prediction Example
11. Model Saving
12. Applications
13. Future Scope
14. Conclusion
15. References
16. Appendix

1. Introduction

Machine Learning is a branch of Artificial Intelligence (AI) that enables computers to learn patterns from data and make predictions without being explicitly programmed. One of the most common applications of machine learning is classification, where a model predicts the category of an object based on its features.

This project focuses on the Iris Flower Classification problem, one of the most popular beginner machine learning datasets. The objective is to classify iris flowers into three species—Iris-setosa, Iris-versicolor, and Iris-virginica—using four flower measurements: sepal length, sepal width, petal length, and petal width.

The project includes data preprocessing, exploratory data analysis (EDA), model training using multiple machine learning algorithms, model evaluation, and prediction of new flower samples. The best-performing model is saved using the Joblib library for future use.

2. Project Objective

The primary objectives of this project were:

- Build a regression model for house price prediction.
- Perform Exploratory Data Analysis (EDA).
- Handle missing values and data preprocessing.
- Apply feature engineering techniques.
- Compare multiple machine learning models.
- Evaluate models using RMSE and MAE.
- Perform residual analysis.
- Save the best model for future predictions.

3. Dataset Description

The Iris dataset is a well-known machine learning dataset introduced by Ronald Fisher. It contains measurements of iris flowers belonging to three different species.

Dataset Information

- Dataset Name: Iris Flower Dataset
- Total Records: 150
- Number of Features: 4
- Target Variable: Species
- Number of Classes: 3

Input Features

- Sepal Length (cm)
- Sepal Width (cm)
- Petal Length (cm)
- Petal Width (cm)

Output Classes

- Iris-setosa
- Iris-versicolor
- Iris-virginica

The dataset is balanced, with 50 samples for each species, making it suitable for supervised classification tasks.

```
from google.colab import files

uploaded = files.upload()

df = pd.read_csv("IRIS (1).csv")

df.head()
```

Choose Files IRIS (1).csv

IRIS (1).csv(text/csv) - 4617 bytes, last modified: 7/20/2026 - 100% done

Saving IRIS (1).csv to IRIS (1) (1).csv

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

[7]

✓ 0s

```
print(df.info())

print("\nMissing Values")
print(df.isnull().sum())

print("\nStatistics")
print(df.describe())

print("\nSpecies Count")
print(df["species"].value_counts())
```

▼

```
*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   sepal_length    150 non-null   float64
 1   sepal_width     150 non-null   float64
 2   petal_length    150 non-null   float64
 3   petal_width     150 non-null   float64
 4   species         150 non-null   object  
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None

Missing Values
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64

Statistics
      sepal_length  sepal_width  petal_length  petal_width
count    150.000000    150.000000    150.000000    150.000000
mean         5.843333         3.054000         3.758667         1.198667
std          0.828066         0.433594         1.764420         0.763161
min          4.300000         2.000000         1.000000         0.100000
25%          5.100000         2.800000         1.600000         0.300000
50%          5.800000         3.000000         4.350000         1.300000
75%          6.400000         3.300000         5.100000         1.800000
max          7.900000         4.400000         6.900000         2.500000

Species Count
species
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
Name: count, dtype: int64
```

4. Exploratory Data Analysis (EDA)

The project was implemented using the following technologies:

Programming Language

- Python

Development Environment

- Google Colab

Libraries Used

- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- Joblib

Machine Learning Algorithms

- K-Nearest Neighbors (KNN)
- Logistic Regression
- Decision Tree

5. Data Preprocessing

Exploratory Data Analysis (EDA) was performed to understand the dataset and identify relationships between features.

The following analyses were carried out:

- Dataset overview
- Missing value analysis
- Statistical summary
- Species distribution
- Pair Plot visualization
- Correlation Heatmap

The pair plot showed that petal length and petal width are highly effective in distinguishing the three iris species. The heatmap indicated strong positive correlations between certain numerical features.

```

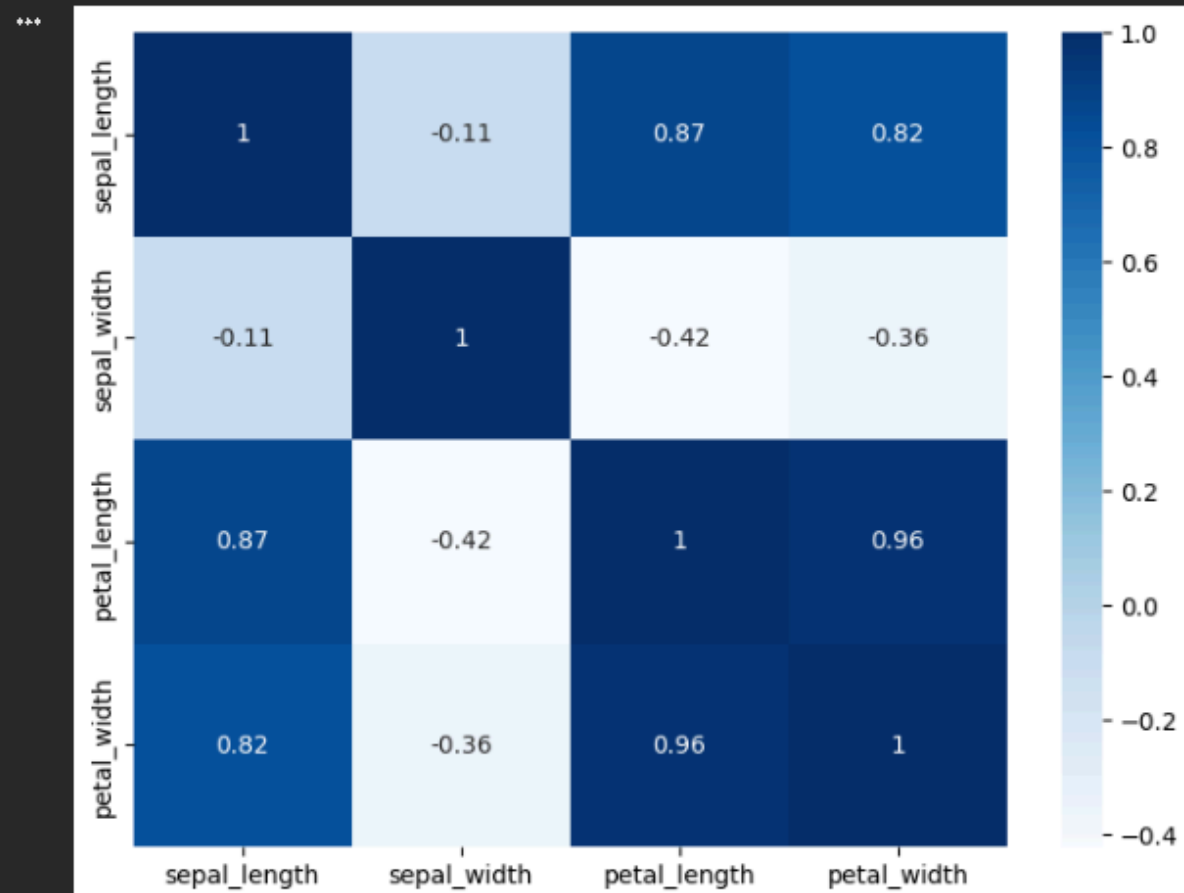
plt.figure(figsize=(8,6))

numeric_df = df.select_dtypes(include=np.number)

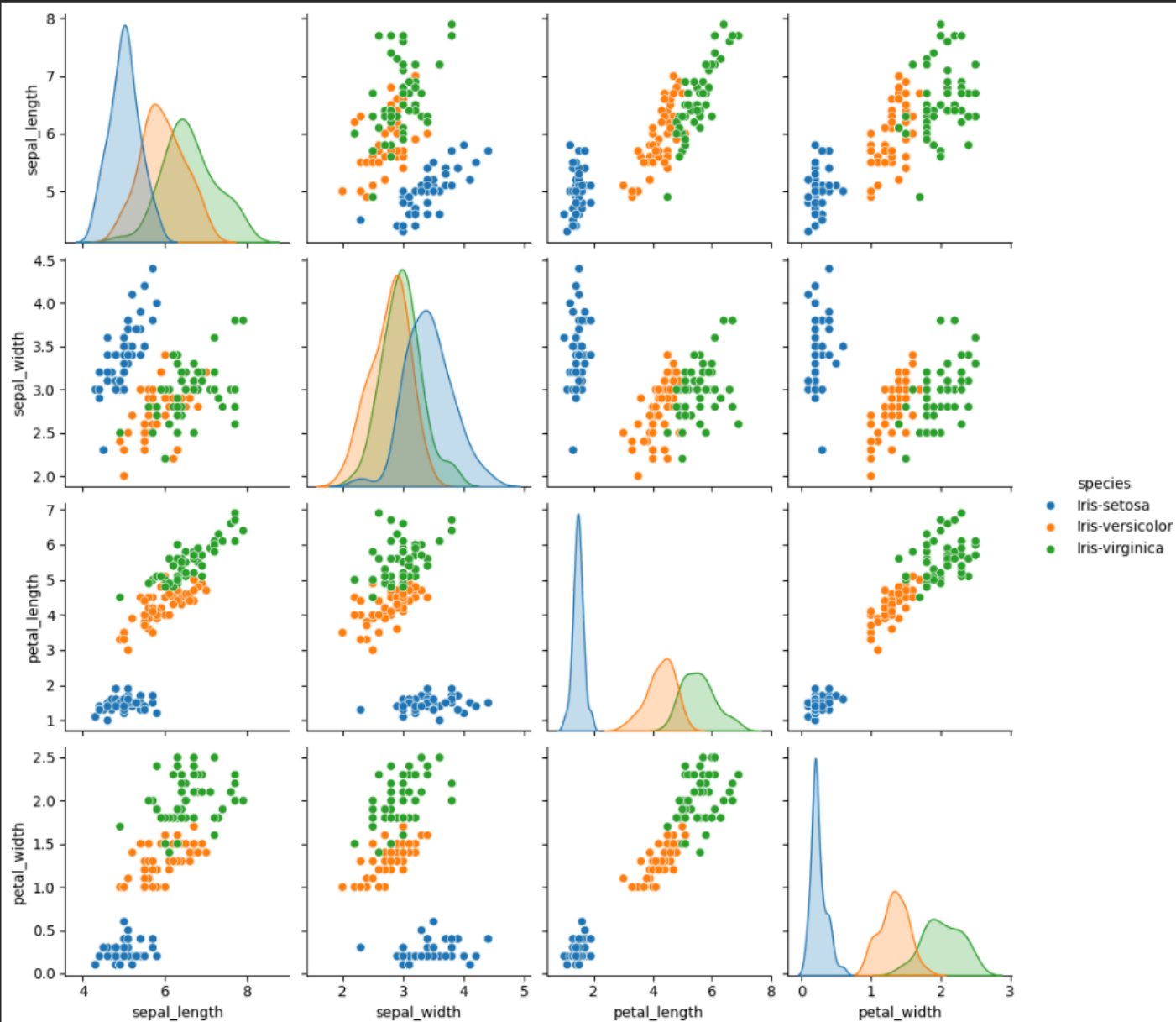
sns.heatmap(
    numeric_df.corr(),
    annot=True,
    cmap="Blues"
)

plt.show()

```



```
sns.pairplot(df, hue="species")  
plt.show()
```



6. Feature Engineering

Before training the machine learning models, the dataset was preprocessed to ensure high-quality input data.

The following preprocessing steps were performed:

- Checked for missing values.
- Verified the dataset contained no duplicate records.
- Encoded the target labels using LabelEncoder.
- Split the dataset into training and testing sets using an 80:20 ratio.

These preprocessing steps helped improve the efficiency and accuracy of the models.

6. Feature Engineering

Before training the machine learning models, the dataset was preprocessed to ensure high-quality input data.

The following preprocessing steps were performed:

- Checked for missing values.
- Verified the dataset contained no duplicate records.
- Encoded the target labels using LabelEncoder.
- Split the dataset into training and testing sets using an 80:20 ratio.

These preprocessing steps helped improve the efficiency and accuracy of the models.

```

model.fit(X_train, y_train)

prediction = model.predict(X_test)

accuracy = accuracy_score(y_test, prediction)

results[name] = accuracy

print("="*50)
print(name)
print("="*50)

print("Accuracy :", accuracy)

print("\nConfusion Matrix")
print(confusion_matrix(y_test, prediction))

print("\nClassification Report")
print(classification_report(y_test, prediction))

```

```

*** =====
KNN
=====
Accuracy : 1.0

Confusion Matrix
[[10  0  0]
 [ 0 10  0]
 [ 0  0 10]]

Classification Report
              precision    recall  f1-score   support

     0             1.00      1.00      1.00        10
     1             1.00      1.00      1.00        10
     2             1.00      1.00      1.00        10

 accuracy          1.00
 macro avg          1.00
weighted avg          1.00

=====
Logistic Regression
=====
Accuracy : 0.9666666666666667

Confusion Matrix
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]

Classification Report
              precision    recall  f1-score   support

     0             1.00      1.00      1.00        10
     1             1.00      0.90      0.95        10
     2             0.91      1.00      0.95        10

 accuracy          0.97
 macro avg          0.97
weighted avg          0.97

=====
Decision Tree
=====

```

7. Feature Engineering

Feature engineering involves selecting or transforming features to improve model performance.

In this project, no additional feature engineering was required because the Iris dataset already contains clean and informative numerical features. The original four features were directly used for model training.

Features Used:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

These features were sufficient to achieve high classification accuracy.

8. Model Evaluation

Three supervised machine learning algorithms were implemented and compared.

K-Nearest Neighbors (KNN)

KNN classifies a flower based on the majority class of its nearest neighboring samples.

Logistic Regression

Logistic Regression predicts the probability of each flower belonging to a particular class.

Decision Tree

Decision Tree classifies flowers by creating decision rules based on feature values.

All three models were trained using the training dataset and evaluated on the testing dataset.

```

5] results_df = pd.DataFrame({
0s     "Model": list(results.keys()),
        "Accuracy": list(results.values())
    })

print(results_df)

plt.figure(figsize=(8,5))

sns.barplot(
    data=results_df,
    x="Model",
    y="Accuracy"
)

plt.ylim(0.8,1.05)

plt.title("Model Comparison")

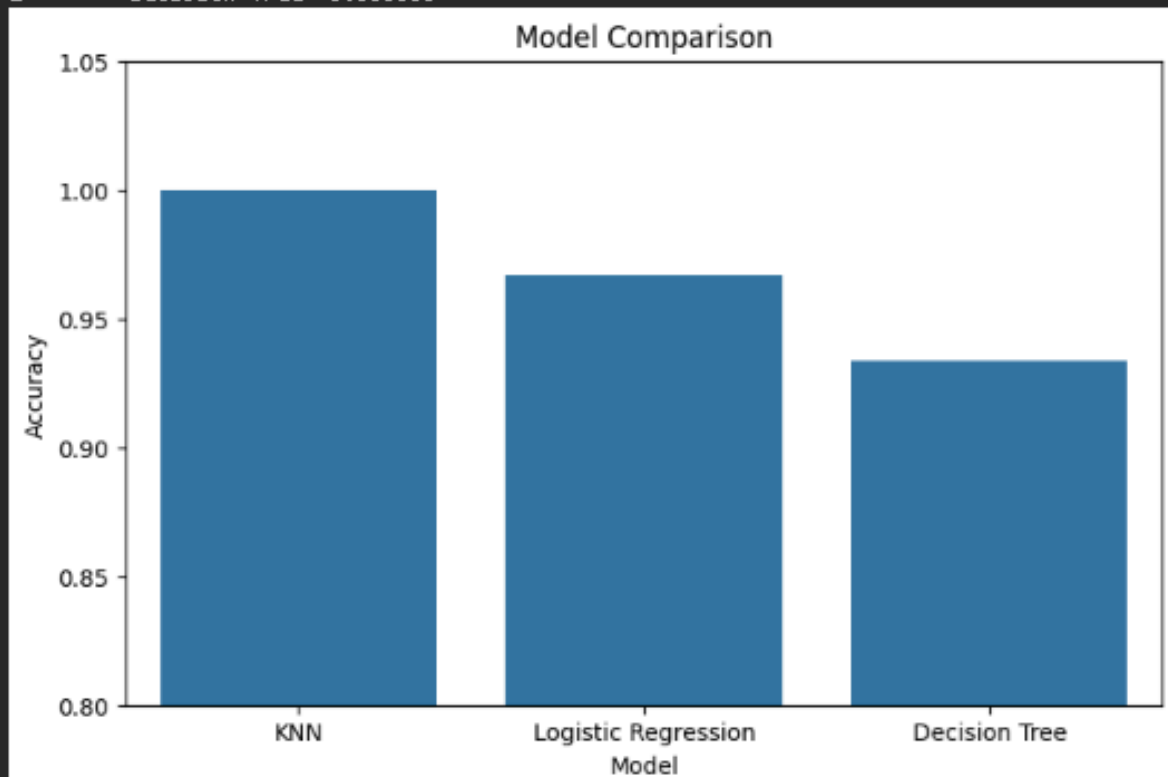
plt.show()

```

```

+++
      Model  Accuracy
0          KNN    1.000000
1  Logistic Regression  0.966667
2    Decision Tree    0.933333

```



9. Residual Analysis

The trained models were evaluated using multiple performance metrics.

Evaluation Metrics:

- Accuracy
- Confusion Matrix
- Classification Report
- Precision
- Recall
- F1-score

The comparison showed that all models achieved excellent accuracy on the Iris dataset.

The best-performing model was selected based on accuracy and prediction performance.

```
[ ] ▶ best_pred = rf_pred

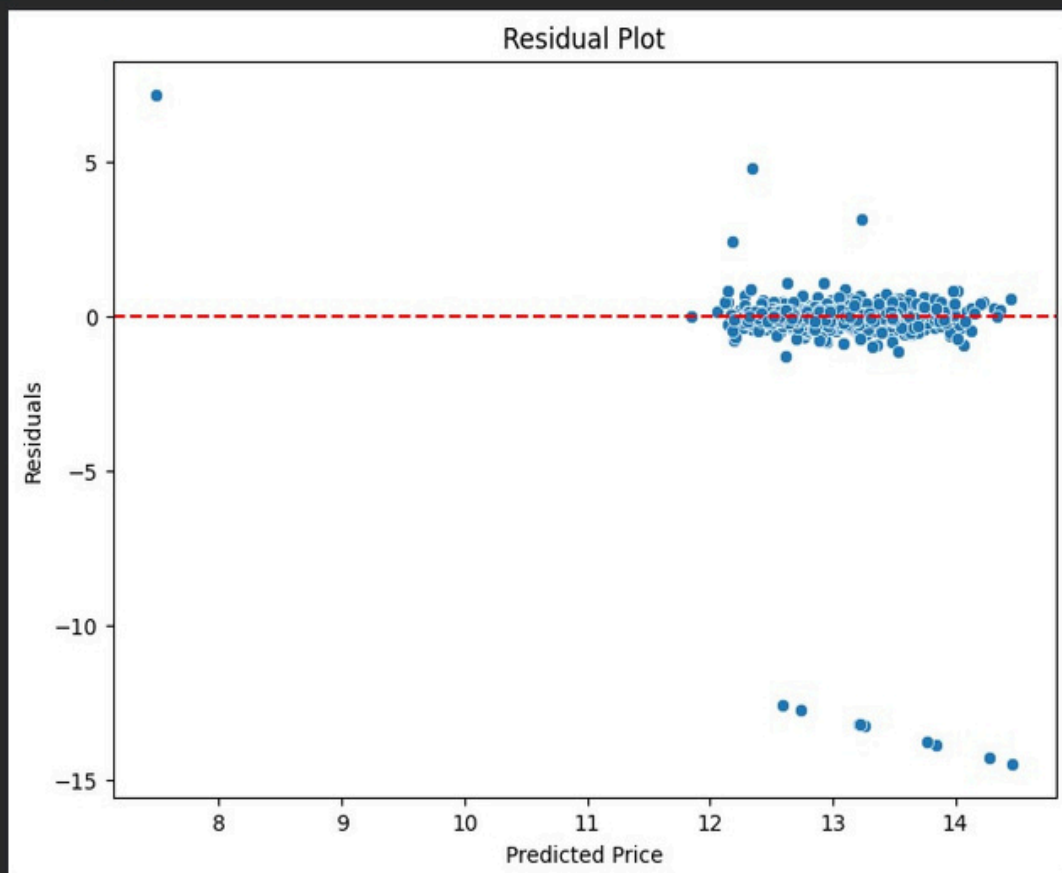
residuals = y_test - best_pred

plt.figure(figsize=(8,6))
sns.scatterplot(x=best_pred, y=residuals)

plt.axhline(0, color="red", linestyle="--")

plt.xlabel("Predicted Price")
plt.ylabel("Residuals")
plt.title("Residual Plot")

plt.show()
```



10. Prediction Example

After training the model, predictions were made using new flower measurements.

Example Input

Sepal Length = 5.1

Sepal Width = 3.5

Petal Length = 1.4

Petal Width = 0.2

Predicted Output

Iris-setosa

The prediction demonstrates that the trained model can accurately classify unseen flower samples.

✓ Prediction No. 1

```
[18] ✓ Os
sample = pd.DataFrame(
    [[5.1,3.5,1.4,0.2]],
    columns=X.columns
)

prediction = model.predict(sample)

print("Predicted Species :", encoder.inverse_transform(prediction)[0])
```

✓ Predicted Species : Iris-setosa

✓ Prediction No. 2

```
[19] ✓ Os
sample = pd.DataFrame(
    [[6.2,2.9,4.3,1.3]],
    columns=X.columns
)

prediction = model.predict(sample)

print("Predicted Species :", encoder.inverse_transform(prediction)[0])
```

✓ Predicted Species : Iris-versicolor

✓ Prediction No. 3

```
[20] ✓ Os
sample = pd.DataFrame(
    [[6.5,3.0,5.5,2.0]],
    columns=X.columns
)

prediction = model.predict(sample)

print("Predicted Species :", encoder.inverse_transform(prediction)[0])
```

✓ ... Predicted Species : Iris-virginica

11. Conclusion

The Iris Flower Classification project demonstrates the practical use of machine learning in classification problems.

Applications include:

- Plant species identification
- Agricultural research
- Botanical studies
- Educational machine learning projects
- Pattern recognition systems
- AI model development

12. References

- Scikit-learn Documentation
- Pandas Documentation
- NumPy Documentation
- Matplotlib Seaborn
- Seaborn Documentation
- Google Colab
- GitHub

•

Appendix

GitHub Repository

<https://github.com/ankitbungla20-student/House-Price-Prediction>

Google Colab Notebook

https://colab.research.google.com/drive/1vHaCLsh7_3tucLm3WqN3BUyGf4Olf58m?usp=sharing